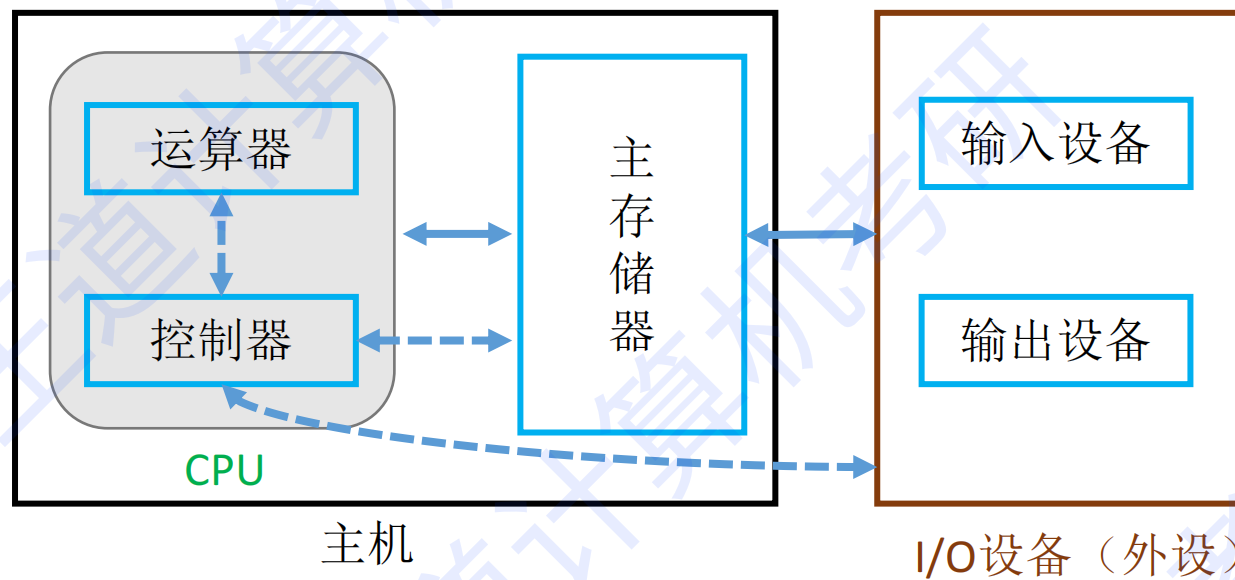


王道考研——计算机组成原理

WWW.CSKAOYAN.COM

第二章 数据的表示和运算

现代计算机的结构



数据如何在计算机中表示？

运算器如何实现数据的算数、逻辑运算？



本节内容

进位计数制

知识总览

进位计数制

十进制、二进制、八进制、十六进制

★ 其他进制 ——> 十进制

★ 二进制、八进制、十六进制之间的相互转换

★ 十进制 ——> 其他进制

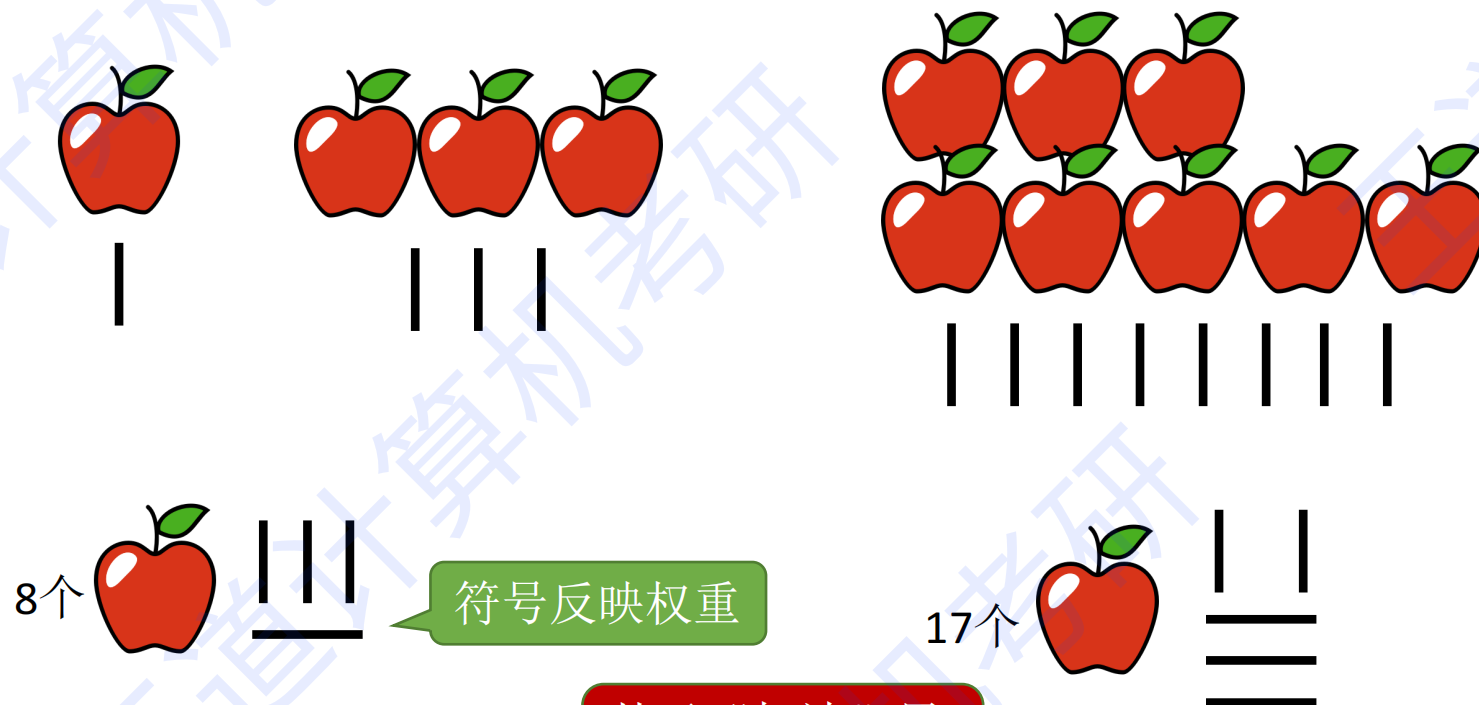
真值和机器数

最古老的计数方法



罗马数字的几种符号与对应权重

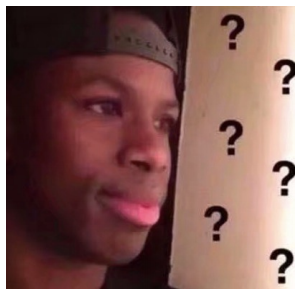
基本字符	I	V	X	L	C	D	M
相应的阿拉伯数字表示为	1	5	10	50	100	500	1000



基于“加法”思想的计数方法

I—1、II—2、III—3、IIII—4 (IV)、V—5
X—10、XI—11、XII—12、XIII—13
MDCLXVI—1666、MDCCCLXXXVIII—1888

十进制计数法



古印度人发明的阿拉伯数字：0, 1, 2, 3, 4, 5, 6, 7, 8, 9

符号反映权重

十进制：

975.36

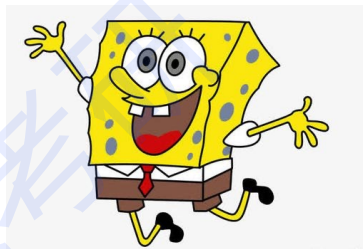
符号所在的位置也反映权重

$$9 \times 100 + 7 \times 10 + 5 \times 1 + 3 \times 0.1 + 6 \times 0.01$$

基于“乘法”思想的计数方法

$$9 \times 10^2 + 7 \times 10^1 + 5 \times 10^0 + 3 \times 10^{-1} + 6 \times 10^{-2}$$

八进制发明者？（误）



十进制： $K_n K_{n-1} \dots K_2 K_1 K_0 K_{-1} K_{-2} \dots K_{-m}$

位权

$$= K_n \times 10^n + K_{n-1} \times 10^{n-1} + \dots + K_2 \times 10^2 + K_1 \times 10^1 + K_0 \times 10^0 \\ + K_{-1} \times 10^{-1} + K_{-2} \times 10^{-2} + \dots + K_{-m} \times 10^{-m}$$

“进位计数制”

有0~9，共十种符号。
逢十进一



推广：r 进制计数法

r 进制: $K_n K_{n-1} \dots K_2 K_1 K_0 K_{-1} K_{-2} \dots K_{-m}$ 位权

$$= K_n \times r^n + K_{n-1} \times r^{n-1} + \dots + K_2 \times r^2 + K_1 \times r^1 + K_0 \times r^0 \\ + K_{-1} \times r^{-1} + K_{-2} \times r^{-2} + \dots + K_{-m} \times r^{-m}$$

基数: 每个数码位所用到的不同符号的个数，r 进制的基数为 r

- ①可使用两个稳定状态的物理器件表示
- ②0, 1 正好对应逻辑值 假、真。方便实现逻辑运算
- ③可很方便地使用逻辑门电路实现算术运算

二进制: 0,1

八进制: 0,1,2,3,4,5,6,7

十进制: 0,1,2,3,4,5,6,7,8,9

十六进制: 0,1,2,3,4,5,6,7,8,9,A,B,C,D,E,F

二进制: $101.1 \rightarrow 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 + 1 \times 2^{-1} = 5.5$

八进制: $5.4 \rightarrow 5 \times 8^0 + 4 \times 8^{-1} = 5.5$

十进制: $5.5 \rightarrow 5 \times 10^0 + 5 \times 10^{-1} = 5.5$

十六进制: $5.8 \rightarrow 5 \times 16^0 + 8 \times 16^{-1} = 5.5$

任意进制→十进制

r 进制: $K_n K_{n-1} \dots K_2 K_1 K_0 \overset{\text{位权}}{K_{-1} K_{-2} \dots K_{-m}}$

$$= K_n \times r^n + K_{n-1} \times r^{n-1} + \dots + K_2 \times r^2 + K_1 \times r^1 + K_0 \times r^0 \\ + K_{-1} \times r^{-1} + K_{-2} \times r^{-2} + \dots + K_{-m} \times r^{-m}$$

二进制: 10010010.110

$$1 * 2^7 + 1 * 2^4 + 1 * 2^1 + 1 * 2^{-1} + 1 * 2^{-2} = 146.75$$

八进制: 251.5

$$2 * 8^2 + 5 * 8^1 + 1 * 8^0 + 5 * 8^{-1} = 169.625$$

十六进制: AE86.1

$$10 * 16^3 + 14 * 16^2 + 8 * 16^1 + 6 * 16^0 + 1 * 16^{-1} = 44678.0625$$

2^{12}	2^{11}	2^{10}	2^9	2^8	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0	2^{-1}	2^{-2}	2^{-3}
4096	2048	1024	512	256	128	64	32	16	8	4	2	1	0.5	0.25	0.125

二进制 $\leftrightarrow$ 八进制、十六进制



如: 1111000010.01101

二进制 $\rightarrow$ 八进制

3位一组, 每组转换成对应的八进制符号

001 111 000 010 . 011 010

1 7 0 2 . 3 2

八进制

二进制 $\rightarrow$ 十六进制

4位一组, 每组转换成对应的十六进制符号

0011 1100 0010 . 0110 1000

3 C 2 . 6 8

十六进制

八进制 $\rightarrow$ 二进制

每位八进制对应的3位二进制

$(251.5)_8 \rightarrow (010\ 101\ 001.101)_2$

十六进制 $\rightarrow$ 二进制

每位十六进制对应的4位二进制

$(AE86.1)_{16} \rightarrow (1010\ 1110\ 1000\ 0110.0001)_2$

各种进制的常见书写方式

二进制—— $(1010001010010)_2$

1010001010010_B

八进制—— $(1652)_8$

十六进制—— $(1652)_{16}$

1652_H

0x1652

十进制—— $(1652)_{10}$

1652_D

十六进制



adj. **hexadecimal** ;

十进制



n. **decimalism**

十进制 → 任意进制

十进制 → 任意进制

r 进制: $K_n K_{n-1} \dots K_2 K_1 K_0 K_{-1} K_{-2} \dots K_{-m}$

$$= K_n \times r^n + K_{n-1} \times r^{n-1} + \dots + K_2 \times r^2 + K_1 \times r^1 + K_0 \times r^0 + K_{-1} \times r^{-1} + K_{-2} \times r^{-2} + \dots + K_{-m} \times r^{-m}$$

如: 75.3 整数部分=75

任一数码位 $K_i < r$

$$\frac{K_n \times r^n + K_{n-1} \times r^{n-1} + \dots + K_2 \times r^2 + K_1 \times r^1 + K_0 \times r^0}{r} = K_n \times r^{n-1} + K_{n-1} \times r^{n-2} + \dots + K_2 \times r^1 + K_1 \times r^0 \dots K_0$$

商 ... 余数

如: 十进制 → 二进制

$$\begin{aligned} 75 \div 2 &= 37 \dots \boxed{1} K_0 \\ 37 \div 2 &= 18 \dots \boxed{1} K_1 \\ 18 \div 2 &= 9 \dots \boxed{0} K_2 \\ 9 \div 2 &= 4 \dots \boxed{1} K_3 \end{aligned}$$

$r = 2$

$$\begin{aligned} 4 \div 2 &= 2 \dots \boxed{0} K_4 \\ 2 \div 2 &= 1 \dots \boxed{0} K_5 \\ 1 \div 2 &= 0 \dots \boxed{1} K_6 \end{aligned}$$

$$75D = 1001011B$$

$$(75)_{10} = (1001011)_2$$

除基	取余
2 75	1
2 37	1
2 18	0
2 9	1
2 4	0
2 2	0
2 1	1
0	

↑ 低位
高位

十进制→任意进制

十进制 → 任意进制

r 进制: $K_n K_{n-1} \dots K_2 K_1 K_0 K_{-1} K_{-2} \dots K_{-m}$

$$= K_n \times r^n + K_{n-1} \times r^{n-1} + \dots + K_2 \times r^2 + K_1 \times r^1 + K_0 \times r^0 + K_{-1} \times r^{-1} + K_{-2} \times r^{-2} + \dots + K_{-m} \times r^{-m}$$

如: 75.3 小数部分=0.3

$$(K_{-1} \times r^{-1} + K_{-2} \times r^{-2} + \dots + K_{-m} \times r^{-m}) \times r = \underbrace{K_{-1} \times r^0}_{\text{整数}} + \underbrace{K_{-2} \times r^{-1} + \dots + K_{-m} \times r^{-(m-1)}}_{\text{小数}}$$

如: 十进制 → 二进制 $r = 2$

$$0.3 \times 2 = 0.6 = \boxed{0} + 0.6 \quad K_{-1}$$

$$0.6 \times 2 = 1.2 = \boxed{1} + 0.2 \quad K_{-2}$$

$$0.2 \times 2 = 0.4 = \boxed{0} + 0.4 \quad K_{-3}$$

$$0.4 \times 2 = 0.8 = \boxed{0} + 0.8 \quad K_{-4}$$

$$0.8 \times 2 = 1.6 = \boxed{1} + 0.6 \quad K_{-5}$$

.....

$$0.3D = 0.01001\dots B$$

乘基	取整	
0.3		
$\times 2$		
0.6	0	高位
$\times 2$		
1.2	1	
0.2		
$\times 2$		
0.4	0	低位
...		

十进制→二进制（拼凑法）



十进制：260.75、533.125

2^{12}	2^{11}	2^{10}	2^9	2^8	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0	2^{-1}	2^{-2}	2^{-3}
4096	2048	1024	512	256	128	64	32	16	8	4	2	1	0.5	0.25	0.125

真值和机器数



15 $\rightarrow$ 1111

8 $\rightarrow$ 1000

+15 $\rightarrow$ 0 1111

-8 $\rightarrow$ 1 1000

真值 机器数

原码、反码、补码、移码

真值：符合人类习惯的数字

机器数：数字实际存到机器里的形式，正负号需要被“数字化”

知识回顾与重要考点

进位计数值

r 进制数 ⊖ 基数=r, 每个数码位可能出现 r 种字符。逢 r 进 1

$$\begin{aligned} & K_n K_{n-1} \dots K_2 K_1 K_0 . K_{-1} K_{-2} \dots K_{-m} \\ &= K_n \times r^n + K_{n-1} \times r^{n-1} + \dots + K_2 \times r^2 + K_1 \times r^1 + K_0 \times r^0 \\ & \quad + K_{-1} \times r^{-1} + K_{-2} \times r^{-2} + \dots + K_{-m} \times r^{-m} \end{aligned}$$

r 进制数 → 十进制 ⊖

r 进制数的数值 = 各数码位与位权的乘积之和

二进制 ↔ 八进制 ⊖

每 3 个二进制位对应一个八进制位

二进制 ↔ 十六进制 ⊖

每 4 个二进制位对应一个十六进制位

注意“补位”

十进制 → r 进制 ⊖

整数部分：除基取余法，先取得的“余”是整数的低位

小数部分：乘基取整法，先取得的“整”是小数的高位

真值和机器数 ⊖

真值：实际的带正负号的数值（人类习惯的样子）

机器数：把正负号数字化的数（存到机器里的样子）

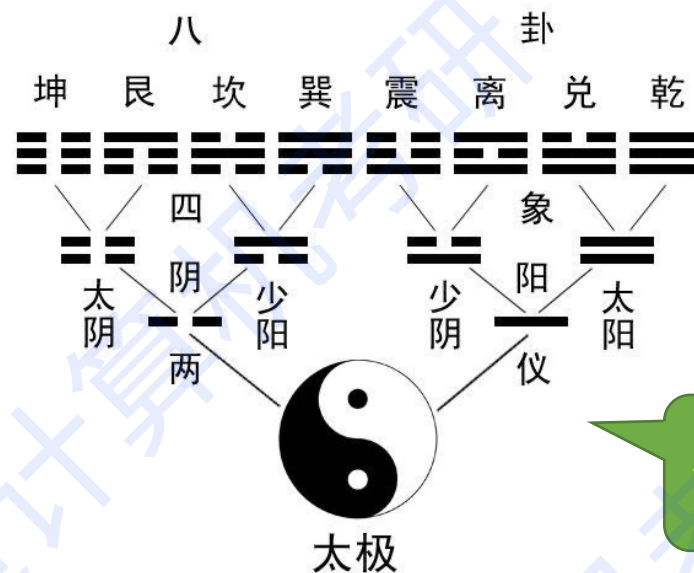
注意：有的十进制小数无法用二进制精确表示，如：0.3

2^{12}	2^{11}	2^{10}	2^9	2^8	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0	2^{-1}	2^{-2}	2^{-3}
4096	2048	1024	512	256	128	64	32	16	8	4	2	1	0.5	0.25	0.125

中国古代的二进制系统



算命 十元一次
你算什么东西?



太极生两仪，两仪生四象，四象生八卦

本节内容

定点数的 表示

定点数 v.s. 浮点数

定点数：小数点的位置固定

Eg: 996.007

——常规计数

浮点数：小数点的位置不固定

Eg: 9.96007×10^2

——科学计数法



二进制的定点数、浮点数也类似

本节总览

定点数的表示

无符号数

有符号数

原码

反码

补码

移码

无符号数的表示

通常只有无符号整数，而没有无符号小数

无符号数：整个机器字长的全部二进制位均为数值位，没有符号位，相当于数的绝对值。

1001 1100B

$$= 1 \times 2^7 + 0 \times 2^6 + 0 \times 2^5 + 1 \times 2^4 + 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 0 \times 2^0$$

=156D

2^{16}	2^{15}	2^{14}	2^{13}	2^{12}	2^{11}	2^{10}	2^9	2^8	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
65536	32768	16384	8192	4096	2048	1024	512	256	128	64	32	16	8	4	2	1

表示范围

8位二进制数： 2^8 种不同的状态

$$\begin{array}{lcl} 0000 \ 0000 & \sim & 1111 \ 1111 \\ 0 & \sim & 255 \end{array} \quad \begin{array}{lcl} = & 1 \ 0000 \ 0000 & - 1 \\ = & 2^8 & - 1 \end{array}$$

$$S_n = a_1 \cdot \frac{1 - q^n}{1 - q}$$

n位的无符号数表示范围为： $0 \sim 2^n - 1$

有符号数的表示

+ 19.75 D = 0 10011.11 B

- 19.75 D = 1 10011.11 B

真值

机器数

8位机器字长



↑
小数点隐藏在此

+ 2.875 D = 0 10.111 B

真值

机器数

8位机器字长



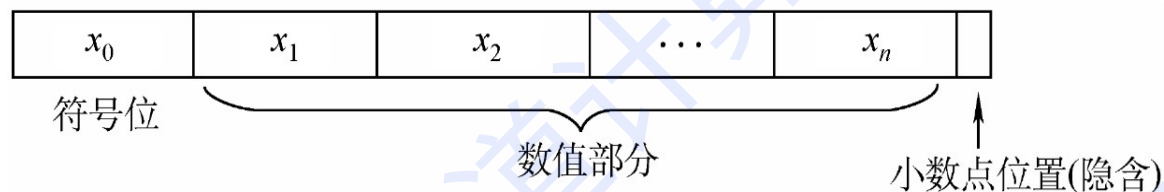
↑
小数点隐藏在此



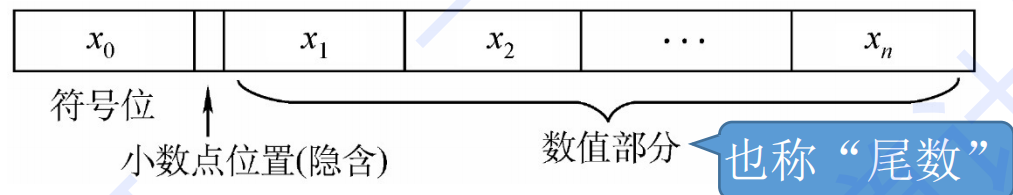
应固定好小数点的位置

有符号数的定点表示

定点整数



定点小数



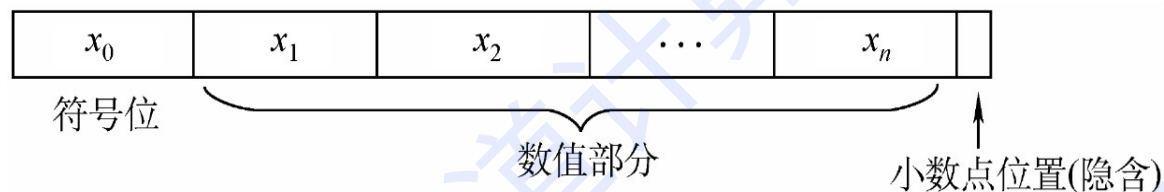
注：可用 **原码**、**反码**、**补码** 三种方式来表示定点整数和定点小数。还可**用移码**表示定点整数。

若真值为 x ，则用 $[x]_{\text{原}}$ 、 $[x]_{\text{反}}$ 、 $[x]_{\text{补}}$ 、 $[x]_{\text{移}}$ 分别表示真值所对应的原码、反码、补码、移码

原码

尾数的位权

定点整数



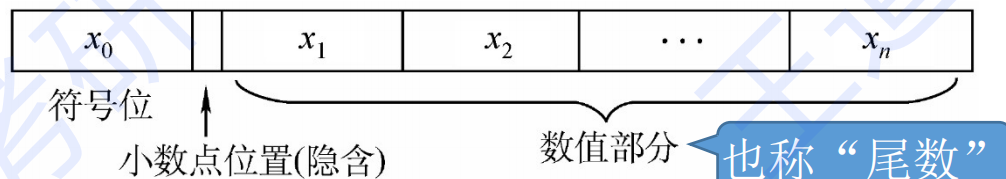
	符	2^6	2^5	2^4	2^3	2^2	2^1	2^0
+19D	0	0	0	1	0	0	1	1
-19D	1	0	0	1	0	0	1	1

常写为: $[x]_{\text{原}} = 1,0010011$

若未指明机器字长, 也可写为: $[x]_{\text{原}} = 1, 10011$

原码: 用尾数表示真值的绝对值, 符号位“0/1”对应“正/负”

定点小数



	符	2^{-1}	2^{-2}	2^{-3}	2^{-4}	2^{-5}	2^{-6}	2^{-7}
+0.75D	0	1	1	0	0	0	0	0
-0.75D	1	1	1	0	0	0	0	0

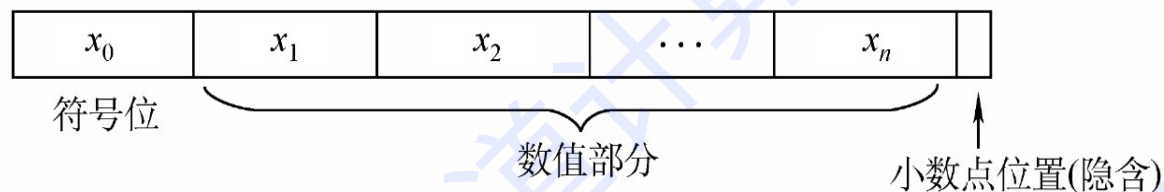
常写为: $[x]_{\text{原}} = 1.1100000$

如: 机器字长为8位

若机器字长为n+1位, 则尾数占n位

原码

定点整数



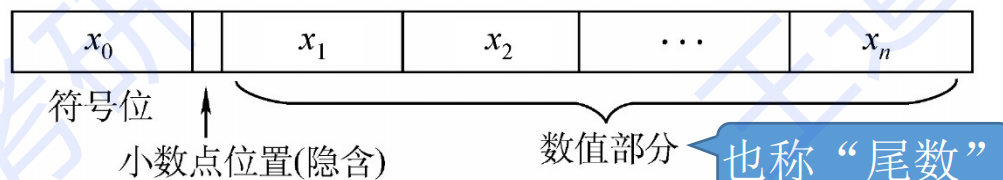
符	2^6	2^5	2^4	2^3	2^2	2^1	2^0
---	-------	-------	-------	-------	-------	-------	-------

若机器字长 $n+1$ 位，**原码整数**的表示范围：
 $-(2^n-1) \leq x \leq 2^n-1$ （关于原点对称）

真值0有 **+0** 和 **-0** 两种形式

原码：用尾数表示真值的绝对值，符号位“0/1”对应“正/负”

定点小数



符	2^{-1}	2^{-2}	2^{-3}	2^{-4}	2^{-5}	2^{-6}	2^{-7}
---	----------	----------	----------	----------	----------	----------	----------

若机器字长 $n+1$ 位，**原码小数**的表示范围：
 $-(1-2^{-n}) \leq x \leq 1-2^{-n}$ （关于原点对称）

真值0有 **+0** 和 **-0** 两种形式

若机器字长为 $n+1$ 位，则尾数占 n 位

反码

反码： 若符号位为0，则反码与原码相同
若符号位为1，则数值位全部取反

x = +19D [x]_原 = 0,0010011

[x]_反 = 0,0010011

x = -19D [x]_原 = 1,0010011

[x]_反 = 1,1101100

定点整数的表示

x = +0.75D [x]_原 = 0.1100000

[x]_反 = 0.1100000

x = -0.75D [x]_原 = 1.1100000

[x]_反 = 1.0011111

定点小数的表示

“反码”只是“原码”转变为“补码”的一个中间状态，实际中并没有什么卵用

若机器字长n+1位，反码整数的表示范围：
 $-(2^n - 1) \leq x \leq 2^n - 1$ （关于原点对称）

真值0有 +0 和 -0 两种形式

[+0]_原 = 00000000

[-0]_原 = 10000000

[+0]_反 = 00000000

[-0]_反 = 11111111

若机器字长n+1位，反码小数的表示范围：
 $-(1 - 2^{-n}) \leq x \leq 1 - 2^{-n}$ （关于原点对称）

真值0有 +0 和 -0 两种形式

补码



补码：正数的补码 = 原码
负数的补码 = 反码末位+1（要考虑进位）

将负数补码转回原码的方法相同：尾数取反，末位+1

x = +19D [x]_原 = 0,0010011
 [x]_反 = 0,0010011
 [x]_补 = 0,0010011

x = -19D [x]_原 = 1,0010011
 [x]_反 = 1,1101100
 [x]_补 = 1,1101101

定点整数的表示

[+0]_原 = 00000000
[+0]_反 = 00000000

[-0]_原 = 10000000
[-0]_反 = 11111111

[+0]_补 = [-0]_补 = 00000000

注意！补码的真值0只有一种表示形式

x = +0.75D [x]_原 = 0.1100000
 [x]_反 = 0.1100000
 [x]_补 = 0.1100000

x = -0.75D [x]_原 = 1.1100000
 [x]_反 = 1.0011111
 [x]_补 = 1.0100000

定点小数的表示

定点整数补码 [x]_补 = 1,0000000 表示 $x = -2^7$

若机器字长n+1位，补码整数的表示范围：
 $-2^n \leq x \leq 2^{n-1}$ （比原码多表示一个 -2^n ）

定点小数补码 [x]_补 = 1.0000000 表示 $x = -1$

若机器字长n+1位，补码小数的表示范围：
 $-1 \leq x \leq 1-2^{-n}$ （比原码多表示一个 -1 ）

移码

移码：补码的基础上将符号位取反。注意：移码只能用于表示整数

$x = +19D$

$[x]_{\text{原}}$	$= 0,0010011$
$[x]_{\text{反}}$	$= 0,0010011$
$[x]_{\text{补}}$	$= 0,0010011$
$[x]_{\text{移}}$	$= 1,0010011$

$x = -19D$

$[x]_{\text{原}}$	$= 1,0010011$
$[x]_{\text{反}}$	$= 1,1101100$
$[x]_{\text{补}}$	$= 1,1101101$
$[x]_{\text{移}}$	$= 0,1101101$

定点整数的表示

$[+0]_{\text{原}} = 00000000$

$[-0]_{\text{原}} = 10000000$

$[+0]_{\text{反}} = 00000000$

$[-0]_{\text{反}} = 11111111$

$[+0]_{\text{补}} = [-0]_{\text{补}} = 00000000$

注意！补码的真值0只有一种表示形式

$[+0]_{\text{移}} = [-0]_{\text{移}} = 10000000$

若机器字长 $n+1$ 位，移码整数的表示范围：
 $-2^n \leq x \leq 2^n - 1$ （与补码相同）

移码

真值(十进制)	补码	移码
-128	1000 0000	0000 0000
-127	1000 0001	0000 0001
-126	1000 0010	0000 0010
...	...	...
-3	1111 1101	0111 1101
-2	1111 1110	0111 1110
-1	1111 1111	0111 1111
0	0000 0000	1000 0000
1	0000 0001	1000 0001
2	0000 0010	1000 0010
3	0000 0011	1000 0011
...	...	...
124	0111 1100	1111 1100
125	0111 1101	1111 1101
126	0111 1110	1111 1110
127	0111 1111	1111 1111

真值增大

移码表示的整数
很方便对比大小

用几种码表示定点整数

行数	机器数	真值(十进制)				
		无符号数	原码	反码	补码	移码
1	0000 0000	0	+0	+0	+0, -0	-128
2	0000 0001	1	+1	+1	+1	-127
3	0000 0010	2	+2	+2	+2	-126
...	...	...	...	...	...	...
126	0111 1101	125	+125	+125	+125	-3
127	0111 1110	126	+126	+126	+126	-2
128	0111 1111	127	+127	+127	+127	-1
129	1000 0000	128	-0	-127	-128	0
130	1000 0001	129	-1	-126	-127	1
131	1000 0010	130	-2	-125	-126	2
...	...	...	...	...	...	...
253	1111 1100	252	-124	-3	-4	124
254	1111 1101	253	-125	-2	-3	125
255	1111 1110	254	-126	-1	-2	126
256	1111 1111	255	-127	-0	-1	127

原码和反码的真值0有两种表示

补码和移码的真值0只有一种表示
补码和移码可以多表示一个负数

练习

定点整数 $x=50$ ，用8位原码、反码、补码、移码表示。

$[x]_{\text{原}} = 00110010$; $[x]_{\text{反}} = 00110010$; $[x]_{\text{补}} = 00110010$; $[x]_{\text{移}} = 10110010$;

定点整数 $x=-100$ ，用8位原码、反码、补码、移码表示。

$[x]_{\text{原}} = 11100100$; $[x]_{\text{反}} = 10011011$; $[x]_{\text{补}} = 10011100$; $[x]_{\text{移}} = 00011100$;

求下列各种码对应的真值：

$[x]_{\text{原}} = 10001101 \rightarrow x=-13$

$[x]_{\text{反}} = 10001101 \rightarrow x=-114$

$[x]_{\text{补}} = 10001101 \rightarrow x=-115$

$[x]_{\text{移}} = 10001101 \rightarrow x=13$

$[x]_{\text{原}} = 00001101 \rightarrow x=13$

$[x]_{\text{反}} = 00001101 \rightarrow x=13$

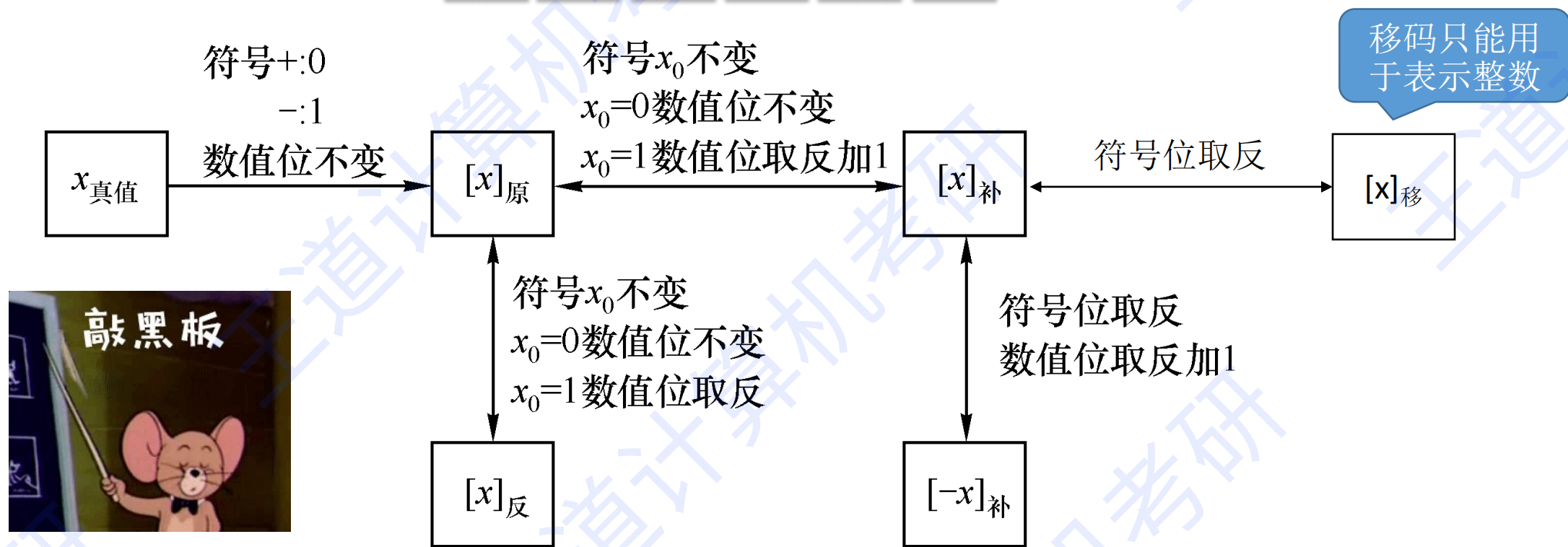
$[x]_{\text{补}} = 00001101 \rightarrow x=13$

$[x]_{\text{移}} = 00001101 \rightarrow x=-115$

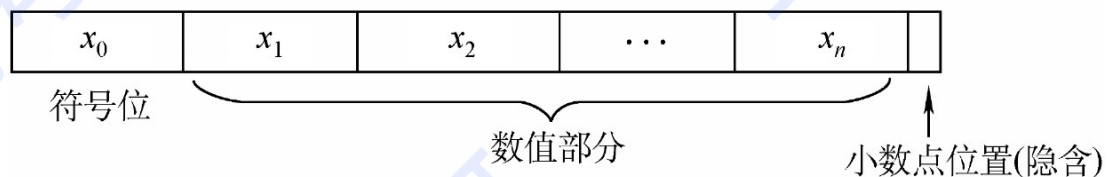
技巧：由 $[x]_{\text{补}}$ 快速求 $[-x]_{\text{补}}$ 的方法

符号位、数值位全部取反，末位+1

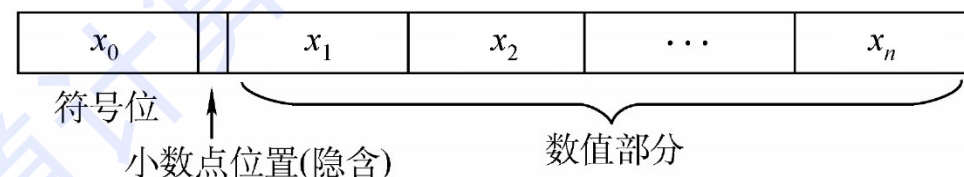
知识回顾



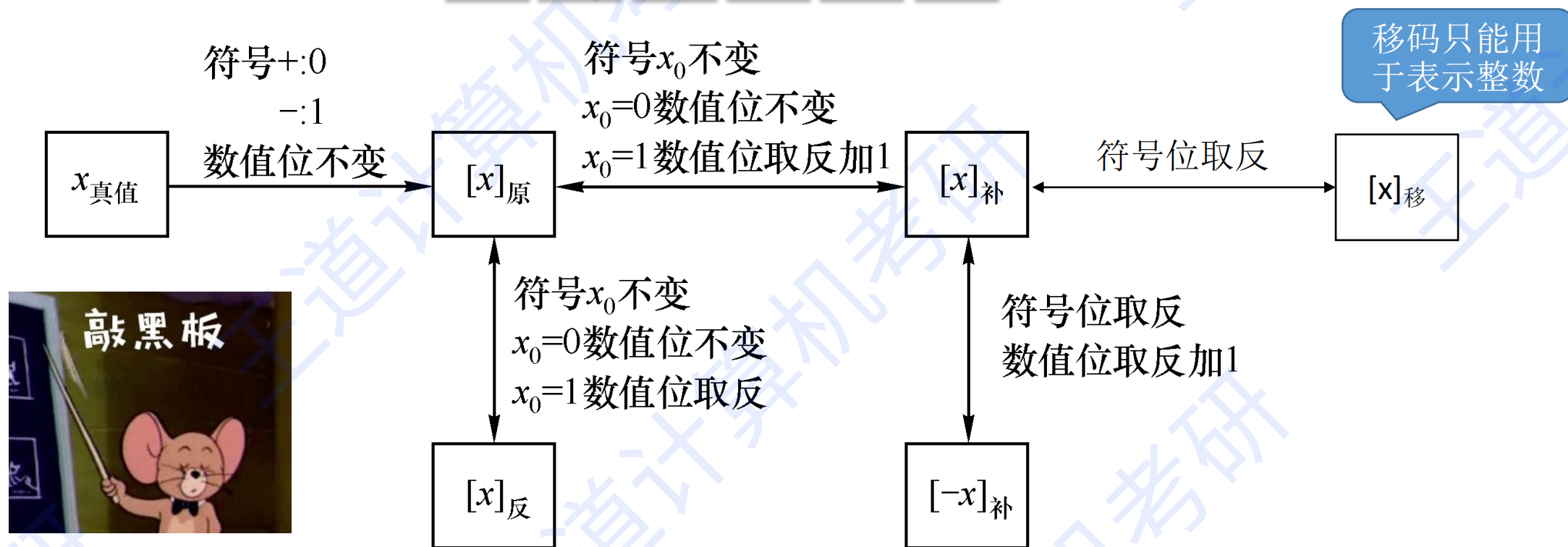
定点整数



定点小数



知识回顾



原码和反码的真值0有两种表示；补码和移码的真值0只有一种表示。若机器字长为 $n+1$ 位，则：

原码和反码

——整数表示范围 $-(2^n-1) \leq x \leq 2^n-1$ ；小数表示范围 $-(1-2^{-n}) \leq x \leq 1-2^{-n}$

补码

——整数表示范围 $-2^n \leq x \leq 2^n-1$ ；小数表示范围 $-1 \leq x \leq 1-2^{-n}$

移码

——整数表示范围 $-2^n \leq x \leq 2^n-1$ ；移码全0真值最小，移码全1真值最大

本节内容

各种码的作用

加减运算

原码表示的有符号数

14
-14
0



+ 00001110
+ 10001110

00001110
- 00001110
00000000



无符号数

00001110
+ 10001110
10011100

14
142
156



能否用加法代替减法

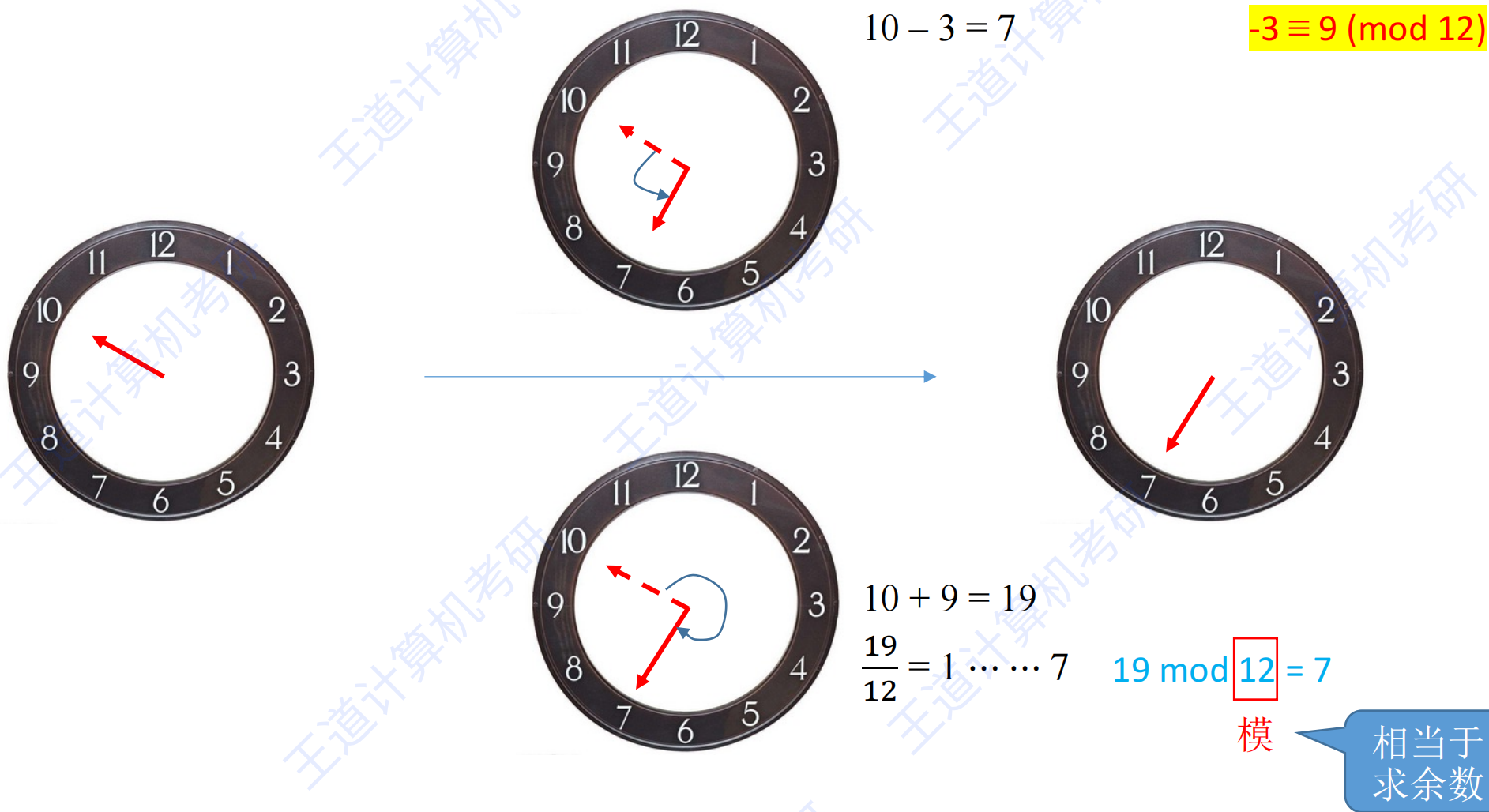


灵光一闪

使用原码运算：
加法——用加法器完成
减法——用减法器完成

.....

用加法代替减法



模运算的性质

带余除法——设 $x, m \in \mathbb{Z}, m > 0$ 则存在唯一决定的整数 q 和 r , 使得:

$$x = qm + r, 0 \leq r < m$$

数论中余数的定义

互为补数

$$-3 = (-1) * 12 + 9$$

二者绝对值之和=模

$$9 = 0 * 12 + 9$$

$$21 = 1 * 12 + 9$$

$$33 = 2 * 12 + 9$$

$$-15 = (-2) * 12 + 9$$

.....

(mod 12) 把所有整数分为 12 类 (余数为 0~11)

mod 12 余数相同的数, 都是同一类, 都是等价的

即 $10 + (-3)$ 、 $10 + 9$ 、 $10 + 21$... 在 (mod 12) 的条件下效果相同

在 (mod m) 的条件下, 若能找到负数的补数, 就可以用正数的加法来等价替代减法

模 - a 的绝对值 = a 的补数

加减运算

有符号数

14
-14
0



$$\begin{array}{r} 00001110 \\ + 10001110 \\ \hline 00001110 \\ - 00001110 \\ \hline 00000000 \end{array}$$

无符号数

14
142
156



任何运算结果在(mod 2⁸)后
只保留最低8位

(mod 2⁸)

$$+ 1,00000000 - 00001110$$

模 - a 的绝对值 = a 的补数

补码——让减法操作
转变为加法操作，节
省硬件成本

$$\begin{array}{r} 00001110 \\ + 11110010 \\ \hline 100000000 \end{array}$$

加减运算

有符号数

88
-66

01011000
- 01000010

(mod 2^8)

+ 1,00000000 - 01000010

模 - a 的绝对值 = a 的补数

01011000
+ 10111110

100010110

22D

补码的作用：

使用补码可将减法操作转变为等价的加法，ALU 中无需集成减法器。
执行加法操作时，符号位一起参与运算

留个坑：溢出的判断？

移码

真值(十进制)	补码	移码
-128	1000 0000	0000 0000
-127	1000 0001	0000 0001
-126	1000 0010	0000 0010
...	...	...
-3	1111 1101	0111 1101
-2	1111 1110	0111 1110
-1	1111 1111	0111 1111
0	0000 0000	1000 0000
1	0000 0001	1000 0001
2	0000 0010	1000 0010
3	0000 0011	1000 0011
...	...	...
124	0111 1100	1111 1100
125	0111 1101	1111 1101
126	0111 1110	1111 1110
127	0111 1111	1111 1111

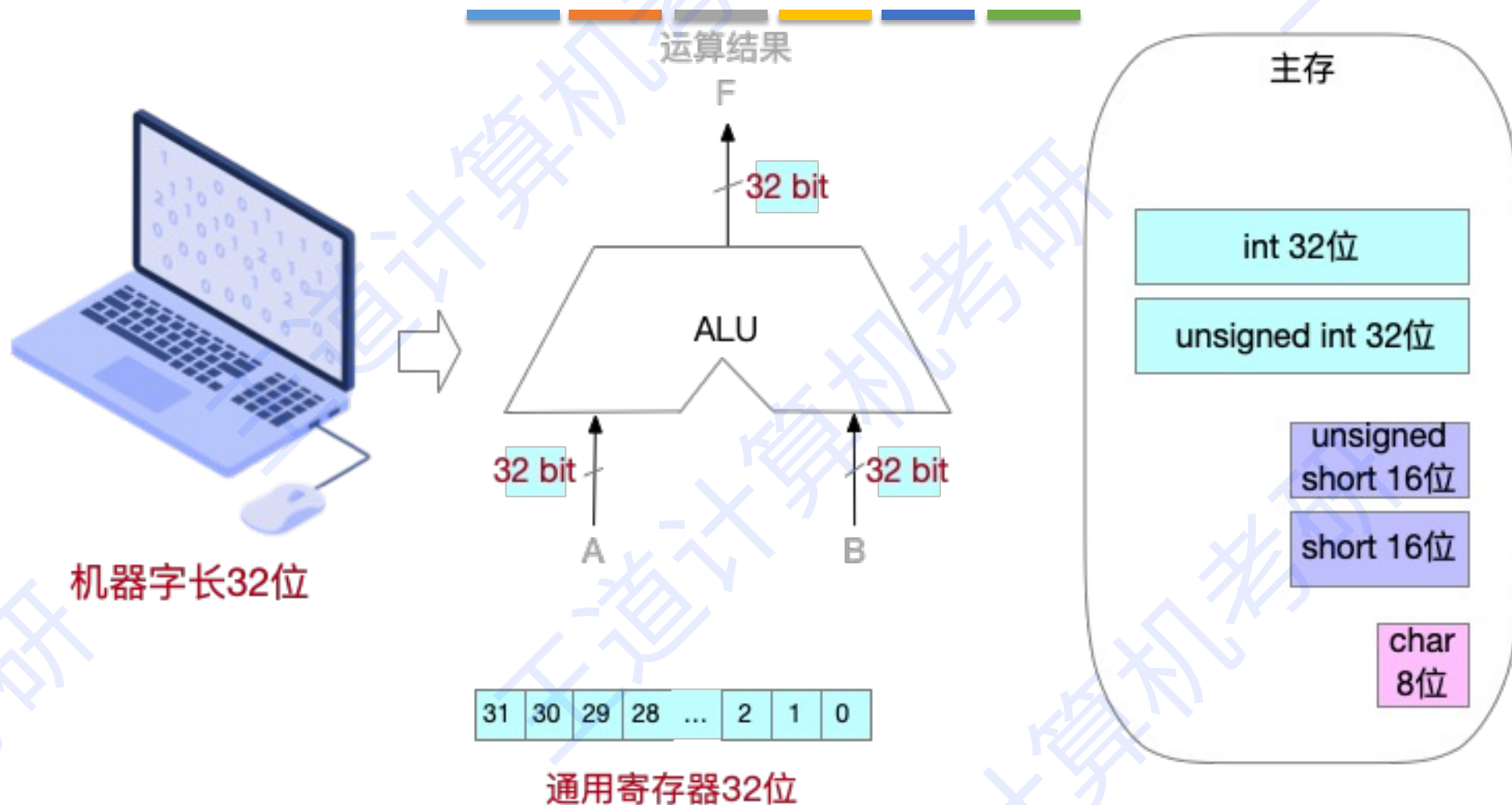
真值增大

移码表示的整数
很方便对比大小

本节内容

零扩展 & 符号扩展

为什么需要对数据进行长度扩展？



- ALU的位数是固定的，运算前可能需把短数据扩展为长数据
- 通用寄存器位数是固定的，把数据存入寄存器时，可能需要进行长度扩展
- 主存内的各种数据长度不一，有时需把短数据扩展为长数据

零扩展、符号扩展

问题：将 8bit 短数据 → 16bit 长数据，多出来的那些位应该怎么填补？

无符号整数 90:

01011010

零扩展

00000000 01011010

无符号整数 166:

10100110

零扩展

00000000 10100110

零扩展：适用于无符号整数，用 0 扩展高位

带符号整数（补码） 90 :

0,1011010

符号扩展

0,00000000 1011010

带符号整数（补码） -90 :

1,0100110

符号扩展

1,11111111 0100110

符号扩展：适用于带符号整数（补码），用“符号位”扩展高位

本节内容

C语言中的 强制类型转换

强制类型转换

void main() { x: 1110 1111 0001 1111 y: 1110 1111 0001 1111 真值61215

```
short x=-4321;                      //short型占用2个字节
unsigned short y=(unsigned short)x;
```

```
int a=165537, b=-34991;            //int型占用4个字节
short c=(short)a, d=(short)b;      //short型占用2个字节
```

```
short x=-4321;
int m=x;
unsigned short n=(unsigned short)x;
unsigned int p=n;
```

} 短整数变长整数:
 符号扩展。

x: 1110 1111 0001 1111
 0xef1f

m: 1111 1111 1111 1111 1110 1111 0001 1111
 0xffffef1f 真值-4321

n: 1110 1111 0001 1111 0xef1f 真值61215

p: 0000 0000 0000 0000 1110 1111 0001 1111
 0x0000ef1f 真值61215

无符号数与有符号数:
不改变数据内容,
改变解释方式。

长整数变短整数:
高位截断, 保留低位。

a: 0x000286a1

c: 0x86a1 真值-31071

b: 0xffff7751

d: 0x7751 真值30545

注: C语言中定点整数
是用“补码”存储的